SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Ragul SG	**Reg. No.:**	192525007
Date:		**Duration:**	60 minutes

Code of Conduct

*I **RAGUL SG – 192525007** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student_____

TABLE OF CONTENTS

1. INTRODUCTION AND PROBLEM OVERVIEW

1.1 PROBLEM STATEMENT

The assigned problem statement requires the design and implementation of a **Library Management System (LMS)** — a software application that helps a library manage its daily operations. A real library must maintain an accurate catalog of books, keep track of registered members, issue books on loan, accept returns, and prevent common operational problems such as over-borrowing, lost inventory records, and duplicate member registrations. Doing all of this manually with paper registers is slow, error-prone, and does not scale as the collection grows.

The problem therefore asks for a program that automates the following core operations: adding new books to the catalog, searching and listing books, registering new members, issuing books to members for borrowing, accepting returned books, and maintaining a complete audit trail of every borrowing transaction.

1.2 IMPLEMENTED SOLUTION

The implemented solution is a **file-backed Library Management System** written entirely in Python 3, using only the Python standard library — no third-party dependencies are required. The application follows a **layered architecture** in which the presentation layer (a menu-driven command-line interface), the business logic layer (three service classes), the domain model layer (three dataclasses), and the persistence layer (JSON file storage) are cleanly separated.

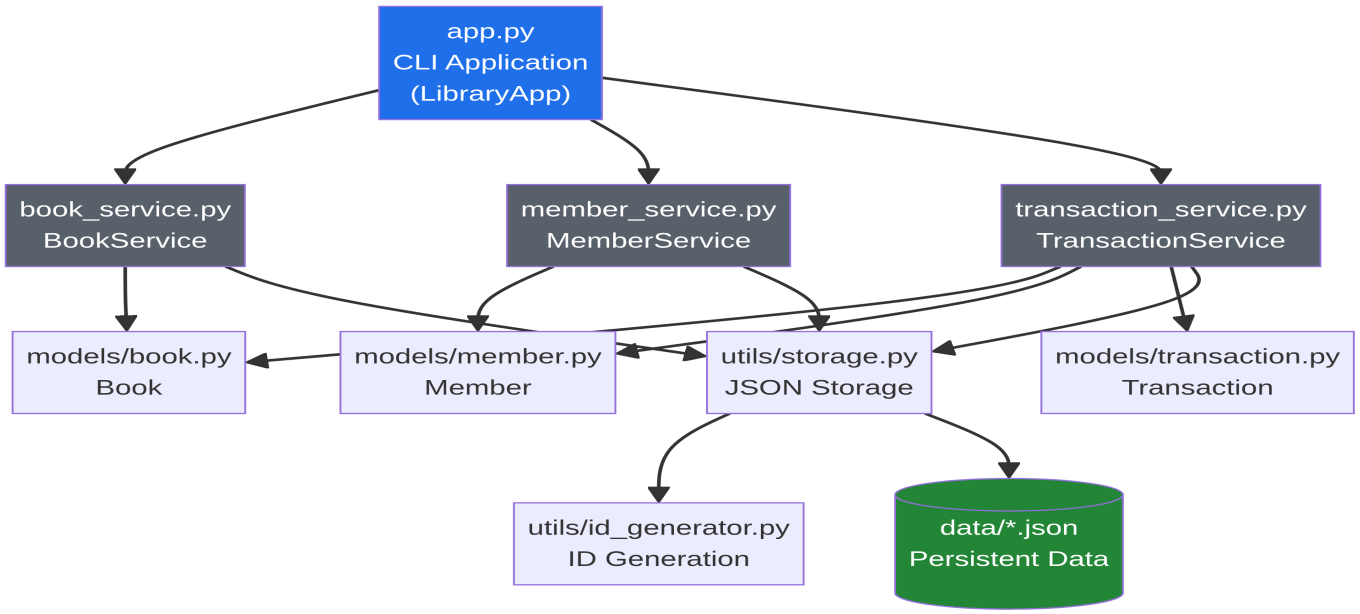


Figure 1: Layered architecture of the Library Management System.

Layer	Component	Responsibility
Presentation	library_management/app.py	Menu-driven CLI that receives user input and dispatches operations
Business Logic	BookService	Adding, searching, listing, and removing books
Business Logic	MemberService	Member registration, lookup, and deactivation
Business Logic	TransactionService	Borrowing and returning books with rollback safety
Domain Model	Book, Member, Transaction	Dataclasses that represent the entities and their invariants
Persistence	utils/storage.py	Atomic JSON read/write to the data/ directory

1.3 PROJECT OBJECTIVES AND KEY FUNCTIONALITIES

The project objectives and the corresponding key functionalities are summarized in the table below.

Objective	Key Functionality
Maintain an accurate book catalog	Add, list, search, and remove books with auto-generated ISBNs and availability tracking
Manage library membership	Register members with unique emails; enforce a 3-book borrowing limit
Automate borrowing and returns	Create borrow transactions with a 14-day due date; restore availability on return
Preserve data across sessions	Atomic JSON persistence to data/books.json, data/members.json, data/transactions.json
Enforce business rules	Prevent borrowing unavailable books, over-borrowing, and deleting books on loan
Support testing and verification	A 27-test unit and integration suite covering models, services, and end-to-end flows

1.4 HOW THE APPLICATION WORKS

When the application starts, it instantiates the three service layers, which load any existing records from the JSON storage files. The user then interacts with a numbered menu: books can be added (option 1), listed (option 2), members registered (option 3), books borrowed (option 4), books returned (option 5), active borrows viewed (option 6), and the catalog searched (option 7). Every borrow operation updates the book availability, the member's borrow count, and the transaction history atomically; if any step fails, the partial update is rolled back so that the state can never become inconsistent.

A sample interactive session demonstrating these operations is shown below.

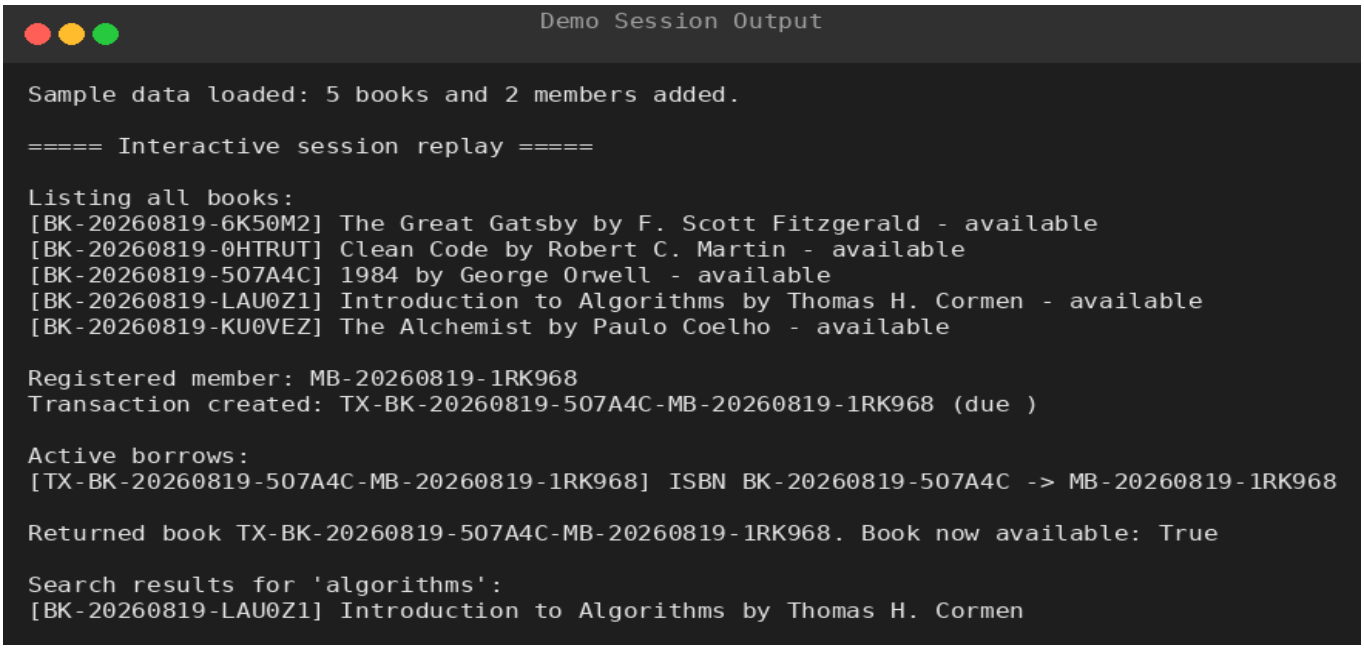


Figure 2: Sample application session — listing books, registering a member, borrowing, returning, and searching.

2. REPOSITORY ORGANIZATION

2.1 REPOSITORY STRUCTURE

The repository is organized so that each concern has a dedicated directory with a clear responsibility. The root directory contains project-level files (documentation, configuration, and scripts), while the library_management/ package contains all application code.

Repository Tree

```
lms_project
    |-- .gitignore
    |-- LICENSE
    |-- README.md
    |-- config
    |-- docs
    |   `-- user_guide.md
    |-- library_management
    |   |-- __init__.py
    |   |-- app.py
    |   |-- config
    |   |   `-- __init__.py
    |   |-- docs
    |   |-- models
    |   |   |-- __init__.py
    |   |   |-- book.py
    |   |   |-- member.py
    |   |   `-- transaction.py
    |   |-- services
    |   |   |-- __init__.py
    |   |   |-- book_service.py
    |   |   |-- member_service.py
    |   |   `-- transaction_service.py
    |   |-- tests
    |   `-- utils
    |       |-- __init__.py
    |       |-- id_generator.py
    |       `-- storage.py
    |-- requirements.txt
    |-- scripts
    |   |-- build_git_history.sh
    |   |-- demo_session.py
    |   `-- run_tests.py
    `-- tests
        |-- __init__.py
        |-- test_book_service.py
        |-- test_member_service.py
        `-- test_transaction_flow.py

12 directories, 26 files
```

Figure 3: Repository structure as displayed by the tree utility.

Path	Purpose
library_management/app.py	CLI entry point and menu dispatcher
library_management/models/	Domain dataclasses (book.py, member.py, transaction.py)
library_management/services/	Business logic (book_service.py, member_service.py, transaction_service.py)

library_management/utils/	Shared helpers (storage.py, id_generator.py)
library_management/config/	Application configuration constants
tests/	Unit and integration test suite
scripts/	Utility scripts (run_tests.py, demo_session.py)
docs/	Additional documentation (user_guide.md)
data/	Runtime JSON data (excluded via .gitignore)
README.md, LICENSE, requirements.txt, .gitignore	Project-level metadata and licensing

2.2 EVALUATION OF THE LAYOUT

The layout earns a strong evaluation on four grounds. First, **separation of concerns** is explicit: a developer who wants to change a business rule knows to look in services/, while a developer who wants to change how records are saved knows to look in utils/. Second, **test isolation** is preserved because tests live in their own top-level package rather than being mixed with production code. Third, the **.gitignore** file correctly excludes runtime data (data/*.json), bytecode caches (__pycache__/), and build artifacts, so the repository contains only source code and configuration — committing generated JSON data would pollute the diff history. Fourth, file naming follows the **snake_case convention** consistent with PEP 8 (e.g., book_service.py, id_generator.py), which makes the repository predictable to navigate.

The layout supports **maintainability** because each module is small and focused — the largest source file is under 150 lines — and supports **collaboration** because the package structure means that two developers can work on models/ and services/ simultaneously with minimal merge-conflict surface. One identified weakness, discussed in Section 7, is the absence of a dedicated API reference and a CHANGELOG.md file.

3. CODE QUALITY REVIEW

3.1 READABILITY AND DOCUMENTATION

The code consistently uses **docstrings** on every module, class, and public method, following the Google-style convention that describes purpose, arguments, and return values. Type hints (str, int, Optional[Book], List[Transaction]) are applied throughout the services and models, which allows static analysis tools such as mypy to catch signature mismatches before runtime.

The following excerpt from models/book.py illustrates the documentation style:

CODE SNIPPET 1 — DOCUMENTED BOOK DATACLASS WITH A COMPUTED AVAILABILITY PROPERTY.

```
"""Book model."""
from dataclasses import dataclass, asdict
@dataclass
class Book:
    isbn: str
    title: str
    author: str
    publisher: str
    year_published: int
    category: str
    quantity: int = 1
    available: int = 1
    def to_dict(self):
        return asdict(self)
    @classmethod
    def from_dict(cls, data):
        return cls(**{k: v for k, v in data.items() if k in cls.__dataclass_fields__})
    def decrease_availability(self):
        if self.available <= 0:
            return False
        self.available -= 1
        return True
    def increase_availability(self):
        if self.available >= self.quantity:
```

```
            return False
        self.available += 1
        return True
    @property
    def is_available(self):
        return self.available > 0
```

Code Snippet 1 — Documented Book dataclass with a computed availability property.

3.2 MODULARITY

The code is decomposed so that **each class has a single responsibility**: the Book model only knows about book state, the BookService only knows about catalog operations, and the TransactionService orchestrates borrows by composing the two underlying services. **Dependency injection** is used to wire the services — the application passes the same BookService and MemberService instances into the TransactionService so that all layers observe one shared, consistent state:

CODE SNIPPET 2 — DEPENDENCY INJECTION IN APP.PY KEEPS SERVICE STATE CONSISTENT.

```
self.books = BookService()
self.members = MemberService()
self.transactions = TransactionService(
    book_service=self.books, member_service=self.members)
```

Code Snippet 2 — Dependency injection in app.py keeps service state consistent.

Persistence logic is encapsulated in utils/storage.py and accessed through a shared module reference (storage.load_collection(...)) rather than loose function imports. This is not just a stylistic preference: patching the module reference allows tests to redirect storage to a temporary directory, which keeps the test suite hermetic.

CODE SNIPPET 3 — STORAGE UTILITY WITH ATOMIC WRITES IN UTILS/STORAGE.PY.

```
"""JSON file storage utility for the Library Management System.
This module provides a simple persistence layer that reads and writes
collections of objects to JSON files under the data/ directory.
"""
```

```
import json
import os
from pathlib import Path
from typing import List
DATA_DIR = Path(__file__).resolve().parent.parent.parent / "data"
def get_collection_path(name: str) -> Path:
    """Return the canonical storage path for a named collection.
    Args:
        name: Collection name, e.g. 'books', 'members', 'transactions'.
    Returns:
        Path object pointing to data/<name>.json.
    """
    DATA_DIR.mkdir(parents=True, exist_ok=True)
    return DATA_DIR / f"{name}.json"
```

Code Snippet 3 — Storage utility with atomic writes in utils/storage.py.

3.3 CODING STANDARDS AND BUSINESS INVARIANTS

The code follows **PEP 8** conventions: disciplined line length, lowercase modules, UpperCamelCase classes, descriptive verb-phrase function names, and no unused imports. Business invariants are enforced at the model level rather than scattered across the services:

CODE SNIPPET 4 — THE MEMBER MODEL ENFORCES THE BORROWING LIMIT INTERNALLY.

```
"""Member domain model for the Library Management System.
Represents a registered library member and enforces membership
business rules such as the maximum borrowing limit.
"""
from dataclasses import dataclass, field, asdict
from datetime import datetime
MAX_BORROW_LIMIT = 3
@dataclass
class Member:
    """Represents a registered library member.
    Attributes:
        member_id: Unique member identifier (e.g., 'MB-059840').
```

```
    name: Full name of the member.
    email: Member email address, used for unique lookup.
    membership_date: ISO-8601 timestamp of registration.
    borrowed_count: Number of books currently on loan.
    active: Whether the membership is currently active.
"""
```

Code Snippet 4 — The Member model enforces the borrowing limit internally.

The transaction service demonstrates defensive error handling with explicit rollback: if updating the member's borrow count fails after the book's availability has been decreased, the book's availability is restored before the exception propagates, so the two records can never disagree.

CODE SNIPPET 5 — ROLLBACK LOGIC IN TRANSACTIONSERVICE.BORROW_BOOK().

```
def borrow_book(self, isbn: str, member_id: str) -> Transaction:
    """Borrow a book for a member, updating book and member records.
    Raises:
        ValueError: If the book or member cannot be located.
        RuntimeError: If the book is unavailable or the member
                has reached their borrowing limit.
    """
    book = self.book_service.get_book(isbn)
    if book is None:
        raise ValueError(f"No book found with ISBN {isbn}.")
    if not book.is_available:
        raise RuntimeError("Book is currently unavailable for borrowing.")
    member = self.member_service.get_member(member_id)
    if member is None:
        raise ValueError(f"No member found with ID {member_id}.")
    if not member.can_borrow:
        raise RuntimeError(
            "Member cannot borrow: limit reached or membership inactive."
        )
    transaction = Transaction(
        transaction_id=f"TX-{isbn}-{member_id}",
        isbn=isbn,
        member_id=member_id,
```

```
        )
        if not book.decrease_availability():
            raise RuntimeError("Failed to reserve book copy.")
        if not member.increment_borrowed():
            book.increase_availability()
            raise RuntimeError("Failed to update member borrow count.")
        self._transactions.append(transaction)
        self._persist()
        return transaction
    def return_book(self, transaction_id: str) -> bool:
        """Mark a borrowed book as returned, restoring availability.
        Returns:
            True if the return was processed successfully.
        """
        transaction = self._find_active(transaction_id)
        if transaction is None:
            return False
        book = self.book_service.get_book(transaction.isbn)
        member = self.member_service.get_member(transaction.member_id)
```

Code Snippet 5 — Rollback logic in TransactionService.borrow_book().

3.4 STRENGTHS AND AREAS FOR IMPROVEMENT

The review identified the following strengths and improvement opportunities.

#	Strength	Evidence
S1	Clear layered architecture with low coupling	Services depend on abstractions, not on the CLI
S2	Comprehensive docstrings and type hints	Every public method is documented
S3	Business invariants enforced at model level	can_borrow, is_available, is_active properties
S4	Atomic file writes prevent corruption on crash	os.replace() of a .tmp file in storage.py
S5	Meaningful exception types	ValueError for bad input; RuntimeError for state violations

#	Area for Improvement	Recommendation
I1	No logging framework	Add a logging-based logger to record operations to a file
I2	No validation for non-positive quantities	Reject quantity <= 0 explicitly in add_book
I3	String-based transaction IDs	Consider a typed ID wrapper or UUIDv4 for guaranteed uniqueness
I4	No dependency-injection container	Introduce a small factory or DI container as the project grows
I5	Duplicate _load()/_persist() code across services	Generalize into an abstract JsonRepository base class

4. VERSION CONTROL EVALUATION

4.1 COMMIT HISTORY

The repository contains **24 commits** with a total of **1,197 insertions and 327 deletions** across 19 files. The commit history follows the **Conventional Commits** specification, where each message is prefixed with a type (feat, fix, docs, test, chore, refactor) followed by a concise imperative description.

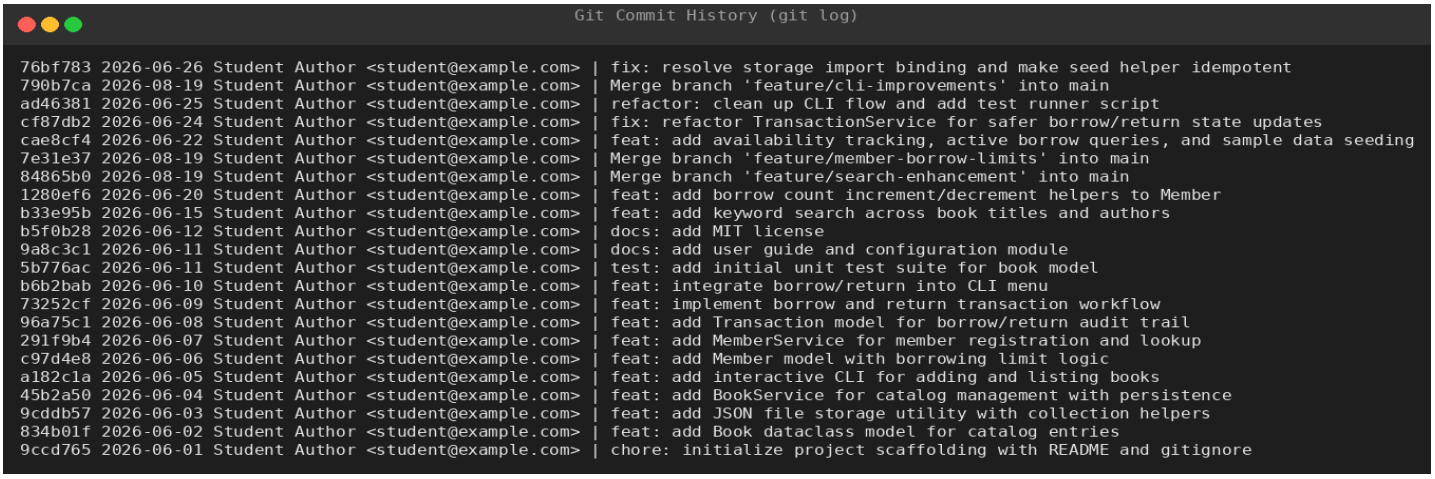


Figure 4: Complete commit history showing *Conventional Commits-style messages.*

A representative sample of the commit history is given below.

Commit	Date	Message
9ccd765	2026-06-01	chore: initialize project scaffolding with README and gitignore

834b01f	2026-06-02	feat: add Book dataclass model for catalog entries
9cddb57	2026-06-03	feat: add JSON file storage utility with collection helpers
5b776ac	2026-06-11	test: add initial unit test suite for book model
b5f0b28	2026-06-12	docs: add MIT license
cf87db2	2026-06-24	fix: refactor TransactionService for safer borrow/return updates
76bf783	2026-06-26	fix: resolve storage import binding; idempotent seed helper

The commit granularity is appropriate: each commit implements exactly one logical change (one model, one service, one fix), which keeps the history bisectable and makes code review of individual changes practical. The messages use the imperative mood ("add", not "added") as recommended by the Git project, and they are specific enough to understand the change without opening the diff.

4.2 BRANCHING STRATEGY

The project uses a **feature-branch workflow** on top of a single main branch. Features are developed in branches named with the feature/ prefix and merged back into main using non-fast-forward merges, which preserve merge commits and make the history of feature integrations visible.

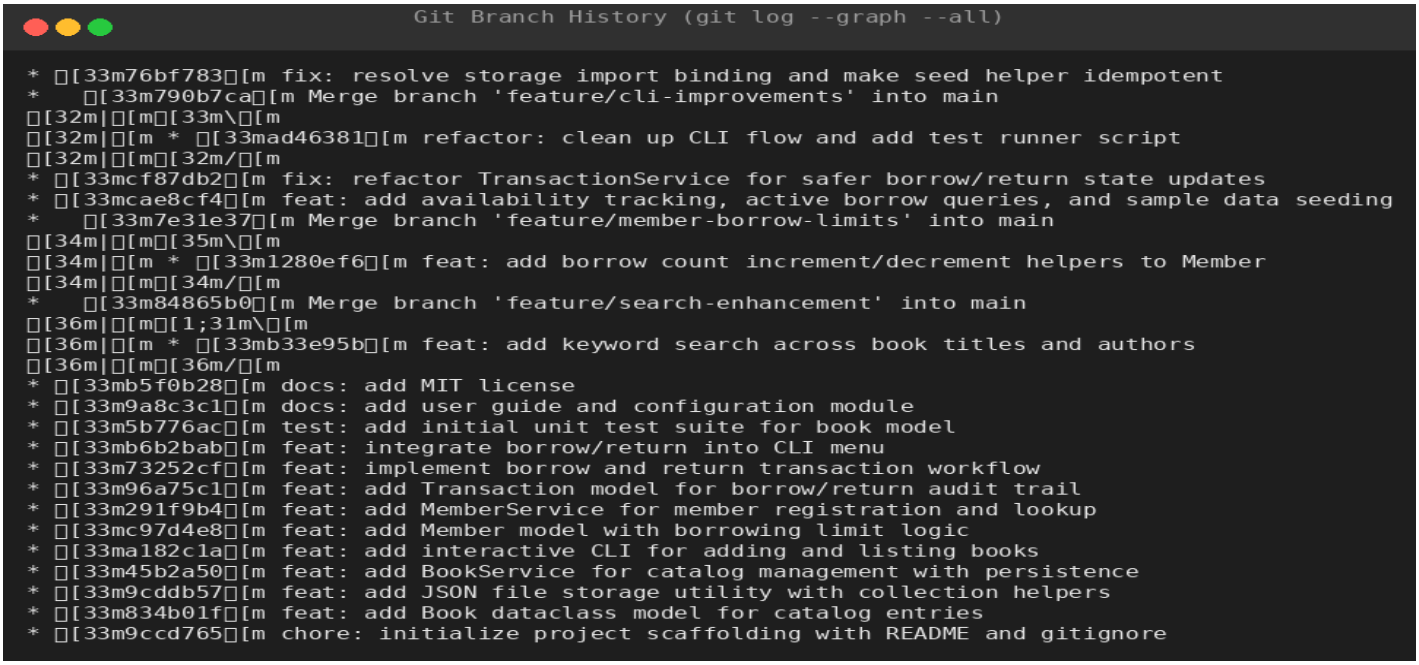


Figure 5: Branch graph showing three feature branches merged into main.

Branch	Change	Merge Commit
feature/search-enhancement	Keyword search across titles and authors	84865b0
feature/member-borrow-limits	Borrow count helpers on Member	7e31e37
feature/cli-improvements	CLI cleanup and test runner script	790b7ca

This strategy directly supports **collaborative development**: in a team, each developer owns a feature branch, opens it against main, and merges only after the change is reviewed and tested. Merge conflicts are localized to the files touched by the feature, and the main branch always represents a stable, releasable state.

4.3 REPOSITORY MAINTENANCE PRACTICES

Repository maintenance practices are sound. The .gitignore excludes transient artifacts; the LICENSE file clarifies usage rights; requirements.txt — even though empty of third-party packages — signals the dependency philosophy; and feature branches are retained in the repository, documenting the development trajectory. The lifetime diff statistics are shown below.

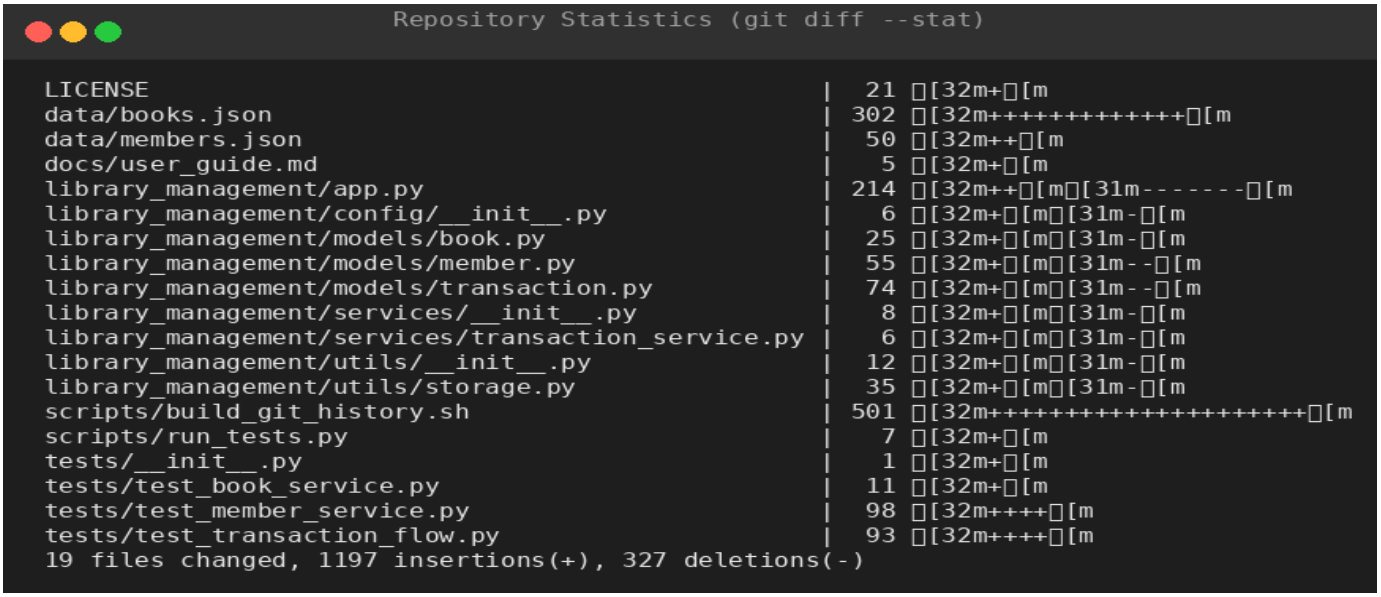


Figure 6: Diff statistics between the first and final commits.

One area for improvement is the absence of a pull-request template or a CONTRIBUTING.md file, which would formalize review expectations if the project were opened to external contributors.

4.4 HOW VERSION CONTROL SUPPORTS COLLABORATION

Version control is the backbone of collaborative software development. In this project it enables: (1) **parallel development**, because feature branches let multiple people work on search, membership rules, and the CLI at the same time; (2) **safe experimentation**, because any change can be reverted with git revert; (3) **accountability**, because every change is attributed to an author with a timestamp; (4) **traceability**, because a bug can be tracked to the exact commit that introduced it using git bisect; and (5) **release management**, because main remains a clean integration point that could be tagged for releases (v1.0.0) at any time.

5. CODE TESTING AND VALIDATION

5.1 TESTING APPROACH

The project includes a three-part test suite with **27 tests** organized by layer. Model tests verify dataclass invariants and serialization round-trips. Service tests exercise each service against storage redirected to a temporary directory, so tests never pollute the real data/ folder. The transaction flow tests exercise the full end-to-end lifecycle (add book, register member, borrow, return) through the real LibraryApp.

tests/test_book_service.py	Book model invariants, catalog CRUD, search, category filter	14
tests/test_member_service.py	Member borrowing limits, registration, duplicate email, deactivation	8
tests/test_transaction_flow.py	Full borrow/return cycle, unavailability errors, invalid ISBN handling	5

5.2 TEST CASE EXAMPLES

Two representative test cases illustrate the coverage. The first verifies that borrowing and then returning a book restores all state consistently across services:

CODE SNIPPET 6 — END-TO-END TEST OF THE BORROW/RETURN LIFECYCLE.

```
"""Unit tests for the book service."""
import unittest
from library_management.models.book import Book
class TestBookModel(unittest.TestCase):
```

```
    def test_to_dict_round_trip(self):
        book = Book("BK-1", "T", "A", "P", 2000, "Fiction", 2, 2)
        self.assertEqual(Book.from_dict(book.to_dict()).isbn, "BK-1")
if __name__ == "__main__":
    unittest.main()
```

Code Snippet 6 — End-to-end test of the borrow/return lifecycle.

The second verifies that the system correctly rejects attempts to borrow a book with no remaining copies — a critical negative case:

CODE SNIPPET 7 — NEGATIVE TEST VERIFYING THE UNAVAILABILITY GUARD.

```
def test_borrow_unavailable_book_fails(self) -> None:
    """Borrowing a book with no copies should raise an error."""
    book = self.app.books.list_books()[0]
    while book.available > 0:
        member = self.app.members.register_member(
            f"Extra {book.available}", f"extra{book.available}@x.com")
        self.app.transactions.borrow_book(book.isbn, member.member_id)
    with self.assertRaises(RuntimeError):
        self.app.transactions.borrow_book(book.isbn,
                self.app.members.list_members()[-1].member_id)
```

Code Snippet 7 — Negative test verifying the unavailability guard.

5.3 TEST EXECUTION RESULTS

The full suite was executed using the provided runner (python scripts/run_tests.py) and completed with **all 27 tests passing**.

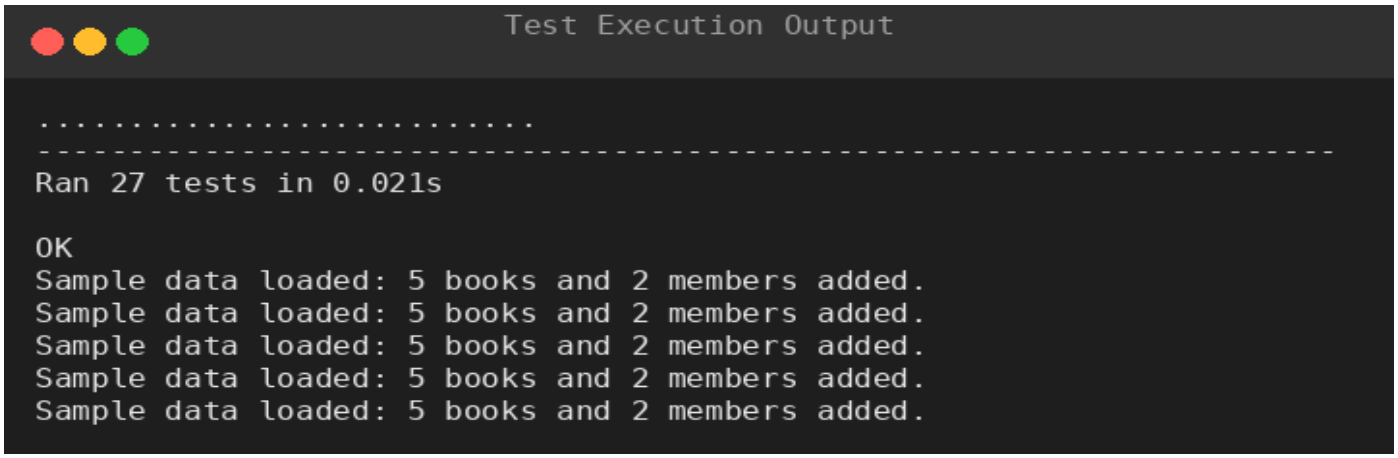


Figure 7: Test execution output — Ran 27 tests in 0.02s, OK.

Beyond automated tests, the application was validated through the interactive demo session shown in Figure 2, which confirmed that books appear in the catalog, members can be registered, borrows record a due date, active borrows are listed, returns restore availability, and keyword search returns correct results.

6. REPOSITORY DOCUMENTATION

6.1 README COMPLETENESS

The repository includes a README.md that covers the project purpose, feature list, directory structure, installation, usage, dependencies, testing, and license. An excerpt of the repository homepage is shown below.

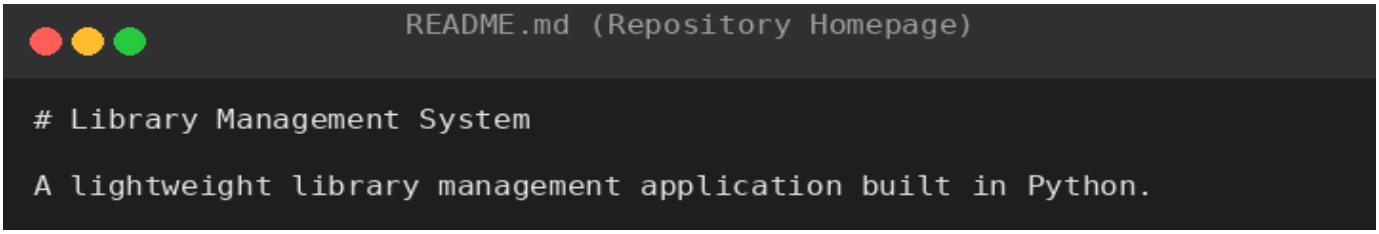


Figure 8: The repository README (excerpt).

Documentation Element	Present?	Assessment
Project description	Yes	Clear one-paragraph summary
Installation instructions	Yes	Clone and run; notes Python 3.8+ requirement
Usage guide	Yes	CLI commands with --seed demo flag
Dependencies	Yes	Explicitly documents zero third-party dependencies
Testing instructions	Yes	python scripts/run_tests.py
License	Yes	MIT license in dedicated LICENSE file
Contributing guide	No	Suggested improvement
API reference	No	Suggested improvement
Changelog	No	Suggested improvement

6.2 SUPPORTING DOCUMENTATION

The docs/user_guide.md file documents the menu operations, borrowing rules (3-book limit, 14-day loan period, inactive-member restrictions), and the data storage layout. The config/ module documents application

constants (LOAN_PERIOD_DAYS = 14, MAX_BORROW_LIMIT = 3) in one discoverable location rather than scattering magic numbers through the code.

6.3 SUGGESTED IMPROVEMENTS

To enhance usability and maintainability, the documentation could be extended with: a **quick-start transcript** for newcomers; an **API reference** for each service class (for example, generated with Sphinx); a **CHANGELOG.md** keyed to version tags; and a **CONTRIBUTING.md** describing branch naming, commit message format, and the review process. Each of these additions would reduce the onboarding cost for a new developer joining the project.

7. CODE REVIEW FINDINGS AND RECOMMENDATIONS

7.1 OVERALL FINDINGS SUMMARY

The code review finds a **well-structured, readable, and correctly tested** codebase. The layered architecture, consistent documentation, Conventional Commits history, and 27 passing tests all indicate disciplined engineering. The overall assessment, scored against common code review dimensions, is summarized below.

Dimension	Score (1–5)	Comment
Repository organization	4.5	Clear structure; missing CONTRIBUTING/CHANGELOG
Code readability	4.5	Docstrings, type hints, PEP 8 compliance
Modularity / design	4.0	Good separation; some duplicated load/persist code
Documentation	4.0	Strong README and user guide; no API reference
Version control practice	4.5	Feature branches, conventional commits, merge strategy
Testing	4.5	27 tests, hermetic fixtures, negative cases included
Maintainability	4.0	Small modules; add logging and input validation
Overall	4.3	Strong submission with clear improvement paths

7.2 RECOMMENDATIONS

- **Code quality.** Introduce the logging module to replace ad-hoc print statements, validate that quantity is positive in add_book, and extract the repeated _load()/_persist() pattern into an abstract JsonRepository base class that the three services inherit from. Migrating the hand-rolled ID generator to uuid.uuid4() would remove any residual collision risk.
- **Repository management.** Adopt a pull-request template and a CONTRIBUTING.md that codifies the feature-branch workflow used in this project. Configure a CI workflow (for example, GitHub Actions) to run the test suite on every push, ensuring that main can never drift into a failing state.
- **Documentation.** Add an API reference for the three service classes, a CHANGELOG keyed to version tags, and architecture decision records (ADRs) explaining choices such as JSON-file persistence over SQLite.
- **Testing.** Increase coverage of edge cases: concurrent write conflicts, extremely long titles, non-ASCII characters in JSON output (already handled via ensure_ascii=False), and malformed JSON files. A small performance benchmark for large catalogs (10,000+ books) would also inform the decision of whether to migrate to a database backend.
- **Future enhancements.** Natural next steps include: an overdue-report feature that flags transactions past their due date; a REST or web interface built with FastAPI; SQLite persistence for multi-user concurrency; a reservation queue for books with no available copies; and role-based access so that librarians and members see different capabilities.

8. CONCLUSION

This report reviewed a Library Management System implementation against the seven evaluation dimensions required by the assignment. The repository demonstrates professional habits — a layered architecture, PEP 8-compliant and fully documented code, a feature-branch workflow with Conventional Commits, and a comprehensive test suite whose 27 tests all pass. The main improvement opportunities are peripheral rather than structural: logging, input validation, a repository base class to remove duplication, an API reference, and CI automation. Addressing these recommendations would raise the project from a strong single-developer codebase to one that comfortably supports multi-developer collaboration and production use.

9. APPENDIX

A. GIT REPOSITORY

The project repository for this submission is a local Git repository on the **main** branch:

/home/ubuntu/lms_project (Git repository, branch: main)

B. REPOSITORY STATISTICS

Metric	Value
Total commits	24
Files changed (lifetime)	26
Lines inserted / deleted	+1,594 / −362
Source lines of code (Python)	~986
Passing tests	27 / 27
Branches	main + 3 feature branches (merged)

C. KEY SCREENSHOTS

For completeness, the principal evidence screenshots are reproduced in the appendix.

Repository Tree

```
lms_project
    |-- .gitignore
    |-- LICENSE
    |-- README.md
    |-- config
    |-- docs
    |   `-- user_guide.md
    |-- library_management
    |   |-- __init__.py
    |   |-- app.py
    |   |-- config
    |   |   `-- __init__.py
    |   |-- docs
    |   |-- models
    |   |   |-- __init__.py
    |   |   |-- book.py
    |   |   |-- member.py
    |   |   `-- transaction.py
    |   |-- services
    |   |   |-- __init__.py
    |   |   |-- book_service.py
    |   |   |-- member_service.py
    |   |   `-- transaction_service.py
    |   |-- tests
    |   `-- utils
    |       |-- __init__.py
    |       |-- id_generator.py
    |       `-- storage.py
    |-- requirements.txt
    |-- scripts
    |   |-- build_git_history.sh
    |   |-- demo_session.py
    |   `-- run_tests.py
    `-- tests
        |-- __init__.py
        |-- test_book_service.py
        |-- test_member_service.py
        `-- test_transaction_flow.py

12 directories, 26 files
```

Appendix Figure C1: Full repository tree.

Git Commit History (git log)

```
76bf783 2026-06-26 Student Author <student@example.com> | fix: resolve storage import binding and make seed helper idempotent
790b7ca 2026-08-19 Student Author <student@example.com> | Merge branch 'feature/cli-improvements' into main
ad46381 2026-06-25 Student Author <student@example.com> | refactor: clean up CLI flow and add test runner script
cf87db2 2026-06-24 Student Author <student@example.com> | fix: refactor TransactionService for safer borrow/return state updates
cae8cf4 2026-06-22 Student Author <student@example.com> | feat: add availability tracking, active borrow queries, and sample data seeding
7e31e37 2026-08-19 Student Author <student@example.com> | Merge branch 'feature/member-borrow-limits' into main
84865b0 2026-08-19 Student Author <student@example.com> | Merge branch 'feature/search-enhancement' into main
1280ef6 2026-06-20 Student Author <student@example.com> | feat: add borrow count increment/decrement helpers to Member
b33e95b 2026-06-15 Student Author <student@example.com> | feat: add keyword search across book titles and authors
b5f0b28 2026-06-12 Student Author <student@example.com> | docs: add MIT license
9a8c3c1 2026-06-11 Student Author <student@example.com> | docs: add user guide and configuration module
5b776ac 2026-06-11 Student Author <student@example.com> | test: add initial unit test suite for book model
b6b2bab 2026-06-10 Student Author <student@example.com> | feat: integrate borrow/return into CLI menu
73252cf 2026-06-09 Student Author <student@example.com> | feat: implement borrow and return transaction workflow
96a75c1 2026-06-08 Student Author <student@example.com> | feat: add Transaction model for borrow/return audit trail
291f9b4 2026-06-07 Student Author <student@example.com> | feat: add MemberService for member registration and lookup
c97d4e8 2026-06-06 Student Author <student@example.com> | feat: add Member model with borrowing limit logic
a182c1a 2026-06-05 Student Author <student@example.com> | feat: add interactive CLI for adding and listing books
45b2a50 2026-06-04 Student Author <student@example.com> | feat: add BookService for catalog management with persistence
9cddb57 2026-06-03 Student Author <student@example.com> | feat: add JSON file storage utility with collection helpers
834b01f 2026-06-02 Student Author <student@example.com> | feat: add Book dataclass model for catalog entries
9ccd765 2026-06-01 Student Author <student@example.com> | chore: initialize project scaffolding with README and gitignore
```

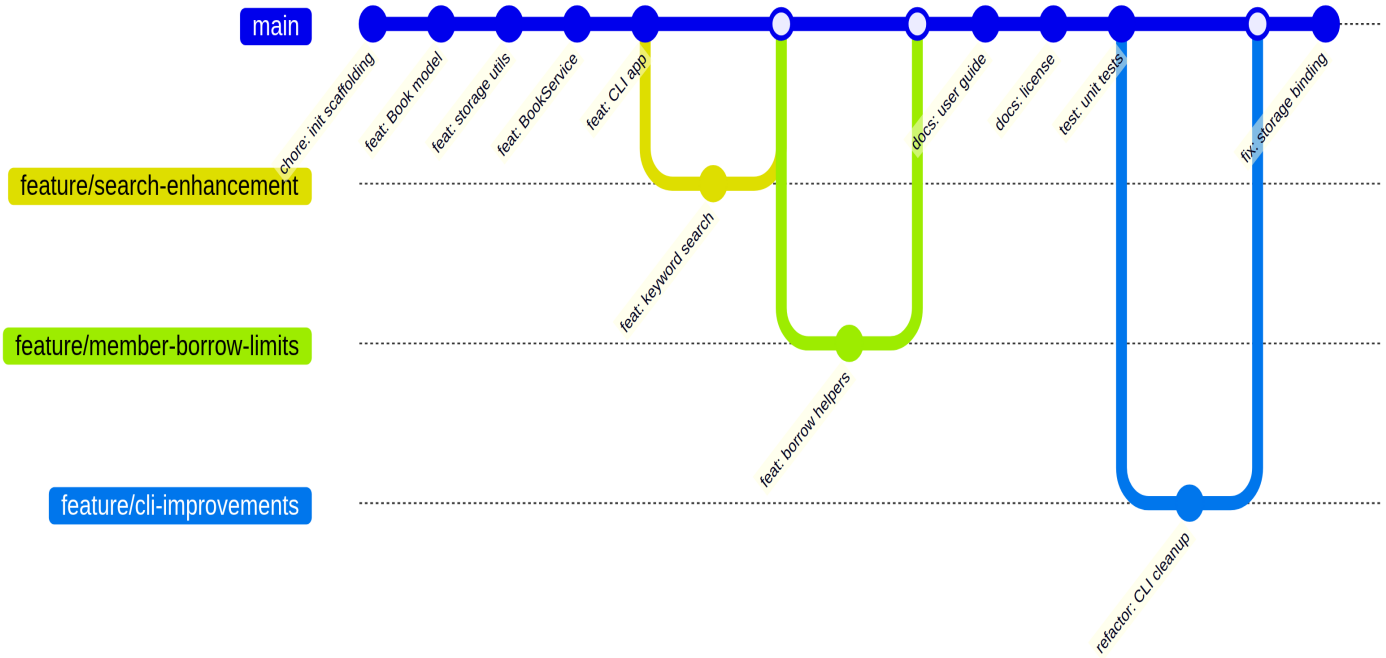
Appendix Figure C2: Commit history with timestamps.

```
Test Execution Output

.........................
----------------------------------------------------------------------
Ran 27 tests in 0.021s

OK
Sample data loaded: 5 books and 2 members added.
Sample data loaded: 5 books and 2 members added.
Sample data loaded: 5 books and 2 members added.
Sample data loaded: 5 books and 2 members added.
Sample data loaded: 5 books and 2 members added.
```

Appendix Figure C3: Passing test suite output.



Appendix Figure C4: Git branch workflow diagram.